Assignment 1 - Git & GitHub Fundamentals

Session 1 - Group B | Individual Lab Report

Full Name: _____

Student ID: _____

Course Code & Section: _____

GitHub Username: _____

---

1. Repository Setup

I initialized the project with `git init -b main`. The `.gitignore` file excludes `local.env` and normal `.log` files. I added an exception for `important.log`, so Git tracks that required file.

2. README & Commit History

The README includes the provided banner, a short project description, technologies, a Git command example, and a link to the official Git documentation. I made six commits with descriptive messages, including the About page commit and the merge commit.

3. Selective Staging

I changed `style.css` and `notes.txt` at the same time. I staged and committed only `style.css`; `notes.txt` stayed modified. I checked the staged change with `git diff --cached` and the unstaged change with `git diff`.

4. Branch & Merge

I created the `feature/about` branch, added an About page, and committed it there. Then I switched back to `main` and merged the feature branch.

5. Remote Changes

A commit records a snapshot in the local repository; a push sends local commits to GitHub. Fetch downloads remote history without changing working files, while pull fetches and integrates remote changes. The GitHub repository, push, online README edit, fetch, and pull steps are still pending because this workspace is not signed in to GitHub. After creating the remote repository, I would push `main`, edit README on GitHub, run `git fetch` and inspect the difference, then use `git pull`.

6. Fair Use of AI

I used AI to interpret the checklist, help draft the README and report, and check the Git workflow. I accepted the suggested project structure and commit sequence, then verified the ignored files, tracked required log, branch history, and staged and unstaged diffs with Git commands. I will review and personalize the report before submission.

7. Sources & Reflection

Source: https://git-scm.com/docs. The hardest concept was keeping staged and unstaged changes straight. If Git behaves unexpectedly, I would first check `git status`, then inspect `git diff` and `git diff --cached`.